

ADMINISTRATORDOKUMENTATION DER ENTERTAINMENT-ERWEITERUNG FÜR IFE

Stand: 03.06.2026, durch Gervithrall Systems

Auftraggeber: Novaris Cabin Systems GmbH

Friedrich-List-Platz 1

01069 Dresden

Ansprechpartner*in: Lea Wagner

E-Mail: lwagner@novaris-cabinystems.de

Telefon: 0351 4620

Auftragnehmer: Gervithrall Systems GmbH

Perlickstraße 1

04103 Leipzig

Ansprechpartner*in: Lucas Rumann

E-Mail: lucasr@gervithrall-systems.de

Telefon: 0351 6482642

INHALTSVERZEICHNIS

1. Übersicht.....	2
2. Systemanforderungen	2
3. Installation	2
4. Build aus dem Quellcode	2
5. Projektstruktur.....	3
6. Konfiguration	3
6.1 Branding (CI-Anpassung)	3
6.2 Sprachkonfiguration.....	4
7. Tests und Qualitätssicherung.....	5
8. Bekannte Einschränkungen.....	5
9. Fehlerbehebung	5

1. Übersicht

IFE Entertainment ist eine offlinefähige Spieleapplikation zur Erweiterung des Inflight-Entertainment-Systems (IFE) von Novaris Cabin Systems GmbH. Die Anwendung wird auf den Sitzmonitoren der Passagiere betrieben und bietet aktuell das Spiel „Vier Gewinnt“ an.

2. Systemanforderungen

- | Java 21 LTS (21.0.x)
- | JavaFX 21.0.2
- | Maven 3.9.15
- | Keine Netzwerkverbindung erforderlich - vollständiger Offline-Betrieb
- | Keine zusätzliche Peripherie erforderlich - Touch oder Mausbedienung

3. Installation

Das Programm wird als ausführbare Fat-JAR-Datei ausgeliefert. Diese enthält den gesamten Code sowie alle JavaFX-Bibliotheken und ist ohne separate JavaFX-Installation lauffähig.

Programmstart

```
java -jar ife.entertainment-[Versionsnummer]-obfuscated.jar
```

4. Build aus dem Quellcode

Voraussetzungen:

- | JDK 21 LTS
- | Maven 3.9.x

Build-Befehl:

```
mvn clean site install
```

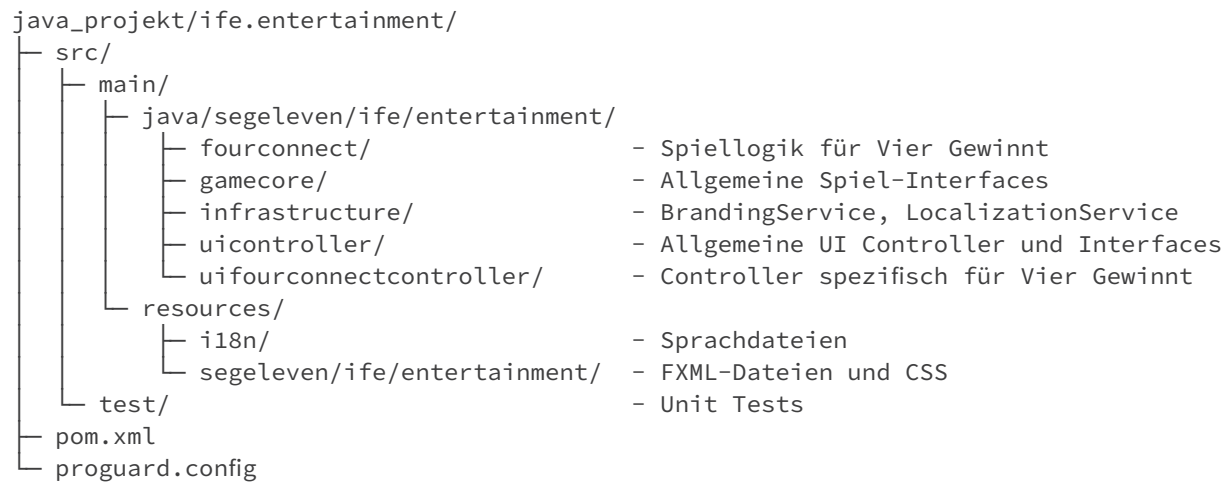
Dieser Befehl führt folgende Schritte automatisch aus:

- 1. Qualitätssicherung** - JUnit Tests werden automatisch ausgeführt
- 2. Code Coverage** - JaCoCo misst die Testabdeckung und erstellt Berichte
- 3. Fat-JAR** - Code und alle JavaFX-Bibliotheken werden zu einer ausführbaren JAR zusammengepackt
- 4. Auto Copy** - Die fertige JAR sowie JavaDocs und Coverage-Berichte werden automatisch in den `final/` Ordner kopiert

Ausgaben nach dem Build:

- | Die fertige JAR-Datei liegt sowohl im `target/` Ordner als auch im `final/` Ordner
- | JavaDocs und Coverage-Berichte liegen ebenfalls im `final/` Ordner
- | Zum Öffnen der Berichte die jeweilige `index.html` im `final/` Ordner starten

5. Projektstruktur



6. Konfiguration

6.1 Branding (CI-Anpassung)

Das Airline-Branding wird über die Klasse `BrandingService.java` verwaltet.

Das System unterstützt verschiedene Themes, wodurch Farben und Branding je nach Airline angepasst werden können.

Folgende Eigenschaften können pro Theme definiert werden:

Eigenschaft	Standardwert
Primärfarbe	#004761
Sekundärfarbe	#555756
Logo-Pfad	/segeleven/ife.entertainment/images/logo_default.png
Airline-Name	Gervithrall Systems

Die Themes werden innerhalb der Methode `setTheme(String themeId)` in `BrandingService.java` (Zeile 139) definiert.

```

switch (themeId) {

    case „gervithrall“:
        this.primaryColor = „#004761“;
        this.secondaryColor = „#555756“;
        this.airlineName = „Gervithrall Systems“;
        this.logoPath = „/segeleven/ife.entertainment/images/logo_gervithrall-systems.png“;
        break;

    case „lufthansa“:
        this.primaryColor = „#05164D“;
        this.secondaryColor = „#FFCC00“;
        this.airlineName = „Lufthansa“;
        this.logoPath = „/segeleven/ife.entertainment/images/logo_lufthansa.png“;
        break;

    default:
        this.primaryColor = „#004761“;
        this.secondaryColor = „#555756“;
        this.airlineName = „Gervithrall Systems“;
        this.logoPath = „/segeleven/ife.entertainment/images/logo_gervithrall-systems.png“;
        break;
}

```

Die definierten Werte werden anschließend automatisch an das CSS-System übergeben und dort als Theme-Variablen verwendet.

Der Wechsel zwischen den verfügbaren Themes erfolgt über die Einstellungen innerhalb der Anwendung (Settings.fxml).

6.2 Sprachkonfiguration

Das Programm unterstützt standardmäßig Deutsch und Englisch. Die Sprache kann in den Einstellungen der Anwendung geändert werden.

Die Sprachdateien liegen unter:

```

src/main/resources/i18n/
├── messages_de.properties  - Deutsche Texte
├── messages_en.properties  - Englische Texte

```

Neue Sprache hinzufügen

5. Neue Properties-Datei erstellen: messages_XX.properties (XX = Sprachcode, z.B. fr für Französisch)

6. Alle Schlüssel aus einer bestehenden Datei übernehmen und übersetzen

7. In LocalizationService.java (Zeile 80) die neue Sprache in getAvailableLocales() eintragen:

```

java
public Locale[] getAvailableLocales() {
    return new Locale[]{Locale.ENGLISH, Locale.GERMAN, Locale.FRENCH};
}

```

8. Neu bauen: `mvn clean site install`

7. Tests und Qualitätssicherung

Unit Tests und Code Coverage werden bei jedem Build automatisch ausgeführt. Der Coverage-Bericht wird automatisch in den `final/` Ordner kopiert - zum Öffnen die `index.html` dort starten.

8. Bekannte Einschränkungen

Die Logo-Pfad-Funktionalität in BrandingService ist aktuell nur vorbereitet und noch nicht aktiv eingebunden.

9. Fehlerbehebung

Programm startet nicht

- | Java-Version prüfen: `java -version` (muss 21 sein)
- | JAR-Datei im `final/` Ordner vorhanden?
- | Versionsnummer im Dateinamen korrekt?

Sprache wird nicht geladen

- | Einstellungen in der Anwendung prüfen
- | Bei anhaltenden Problemen das Entwicklungsteam kontaktieren

Build schlägt fehl

- | Maven Version prüfen: `mvn -version`
- | JDK 21 installiert?
- | `mvn clean install` erneut ausführen

Coverage-Bericht nicht sichtbar

- | `mvn clean site install` ausführen
- | `index.html` im konfigurierten Coverage-Ausgabeordner öffnen

JavaDocs nicht sichtbar

- | `mvn clean site install` ausführen
- | `index.html` im konfigurierten JavaDocs-Ausgabeordner öffnen